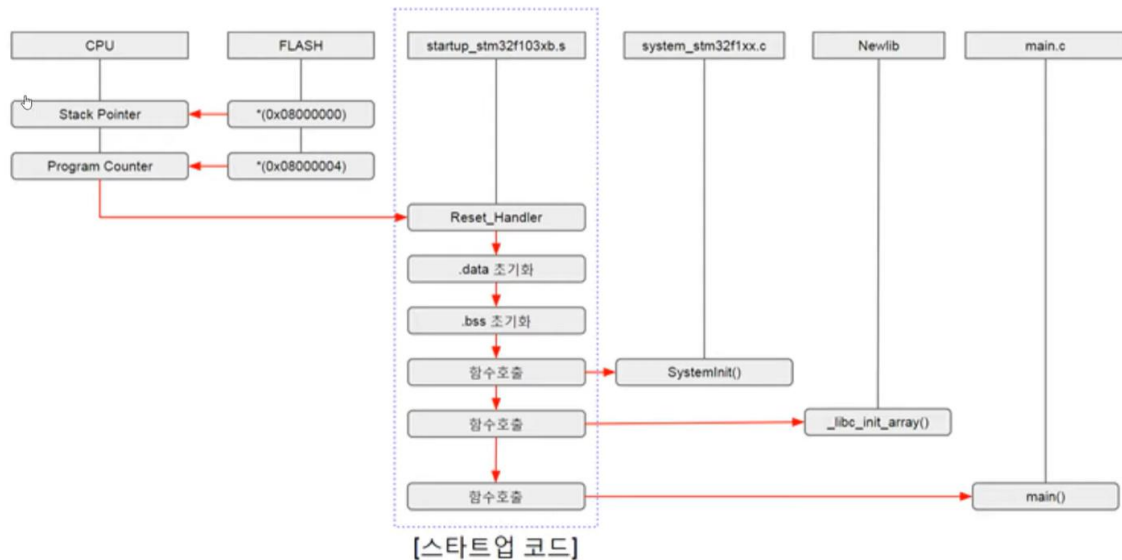


전원 인가 후 메인함수가 가장 먼저 실행된다고 알고 있으나, MCU전원 인가후 메인함수 호출 전까지 startup code가 실행되어 특정 기능들이 실행된다. 대부분 어셈블리어로 이뤄져 있다.



FLASH의 시작주소 값을 읽어 Stack Pointer에 저장하여 초기화 한다. 다음 4바이트 주소를 PC에 저장한다. 그 주소는 스타트업 코드의 Reset_Handler함수가 실행된다. 또한 startupcode의 실행 시작이기도 하다. data섹션에서 초기값이 있는 전역변수를, bss는 초기값이 없는 전역변수를 0으로 클리어한 뒤 초기화한다. bss초기화 과정이 없다면 초기값이 없는 전역변수에 쓰레기값이 들어간다. 또한 다른 메모리 영역에 전역변수를 할당하면 역시 쓰레기값이 저장된다. 따라서 그런 경우 클리어를 위해선 초기화를 직접해줘야 한다. SystemInit함수를 호출한다. 라이브러리 함수를 호출하는 과정은 c++에서 클래스 생성자를 호출하기 위해 필요하며 c언어를 사용하는 경우 필요없다. 마지막으로 메인함수로 넘어간다.

이곳에는 인터럽트 벡터 테이블이 정의되어 있다. 일종의 함수 포인터배열이다. 인터럽트 발생 시 CPU는 테이블을 참고하여 실행할 함수의 주소를 테이블에서 읽는다. isr_vector의 주소는 링커 스크립트에 정의되어 있는데, FLASH의 시작주소에 저장되어 있다. _estack은 RAM의 마지막 주소에 할당된다.

<Reset Handler>

`ldr sp = _estack` 메모리에서 레지스터에 값을 저장한다. estack의 주소값을 sp레지스터에 저장한다. 스택 포인터의 실제 값은 map파일에서 찾을 수 있다.

CopyDataInit과 LoopCopyDataInit은 초기값을 지정한 전역변수를 설정해주는 함수들이다.

movs r1 #0 r1에 0을 저장한다

b LoopCopyDataInit 해당함수를 호출하여 실행한다.

(LoopCopyDataInit) ldr r0 = sdata RAM의 시작부분에 위치한 data섹션의 시작주소인 sdata의 주소값을 r0에 저장한다.

ldr r3 = edata data섹션의 마지막주소인 edata의 주소값을 r3에 저장한다

이때 전역변수들의 초기값은 FLASH에 저장되어 있다. FLASH의 시작주소는 sidata에 저장된다.

add r2, r0, r1 r2 = r0 + r1

cmp r2 r3 r2와 r3를 비교하고 뒤의 조건판단에 사용한다.

bcc CopyDataInit r2가 r3보다 작다면 해당함수로 이동하여 실행한다.

(CopyDataInit)

ldr r3 = _sidata Flash의 시작주소인 데이터섹션의 시작주소를 r3에 저장한다.

ldr r3, [r3,r1] r3 = *(r3+r1) 해당 플래시의 데이터를 r3에 저장한다.

str re, [r0,r1] *(r0 + r1) = r3 저장한 데이터를 RAM에 저장한다

adds r1, r1, #4 이후 다음 FLASH주소에 접근하기 위해 주소값을 증가시킨다.

다시 LoopCopyDataInit이 실행되어 증가된 r1으로 메모리 영역을 검사하여 Flash의 전역변수 데이터를 RAM으로 복사하는 과정을 반복한다.

ldr r2, = _sbss 위 과정이 끝나면 r2에 초기값이 없는 전역변수 섹션의 시작주소를 r2에 저장한다. RAM영역에만 존재한다.

ldr r3, = _ebss bss마지막 주소를 r3에 저장한다. 이후 위와 같은 주소공간 비교를 통해 FillZeroBss로 이동한다.

movs r3 #0 r3 = 0

str r3, [r2], #4 *r2 = r3 초기값이 없는 전역변수를 0으로 클리어 한다. 그리거 r2를 증가시켜 RAM의 다음주소를 저장한다. bss영역을 전부 0으로 채울때까지 위 과정을 반복한다.

마지막으로 systemInit, _libc_init_array를 호출한 뒤 복귀하고 메인함수를 호출한다.